

SECURITY MASTER PLAN & ROADMAP

ft_transcendence (v21.2) — Role Responsibilities, Mandatory Audit & Skills Curriculum

1. Executive Overview & Role Objectives

As the **Security Agent** for your *ft_transcendence* team, your primary mandate is to establish a robust defense-in-depth architecture, enforce all mandatory security constraints defined in the subject, implement dedicated cybersecurity modules, and protect user data. Security is not an afterthought—it must be integrated into every phase of development from Docker containerization to frontend input validation.

Key Security Objective: *Ensure zero critical vulnerabilities, full compliance with subject rules (HTTPS, Vault, WAF, password hashing, input validation), and empower yourself with industry-standard cybersecurity competencies.*

2. Mandatory Subject Security Requirements

The *ft_transcendence* subject explicitly specifies several mandatory security items in Chapter III (Section III.2 & III.3). Failing any of these will result in project rejection:

- **HTTPS Enforcement (Sec III.3):** ANY connection to the backend from a browser, script, or external API MUST use HTTPS (TLS/SSL). Connections inside backend containers (internal Docker network) may be HTTP. You must set up SSL certificates (e.g. mkcert for local dev, Let's Encrypt or Nginx reverse proxy with TLS).
- **Environment Secrets & Credentials (Sec III.3):** API keys, credentials, JWT secrets, and DB strings MUST be stored in a local .env file ignored by Git. You must provide a clean .env.example template with no real secrets committed to the repository.
- **Secure User Password Hashing (Sec III.3):** Standard authentication must use properly hashed and salted passwords. Plaintext passwords or legacy algorithms (MD5, SHA1) are strictly prohibited. Use modern password hashing standard (Argon2id or bcrypt with cost factor >= 12).
- **Double-Sided Input Validation (Sec III.3):** All form inputs and API payloads MUST be validated on BOTH the frontend (UX & quick feedback) and backend (security boundary). Never rely on client-side validation alone.
- **Containerized Security (Sec III.2):** The deployment must run with a single Docker command (e.g. docker compose up --build). Containers must not run as root unnecessarily, and secrets must not be baked into Docker images.
- **Privacy Policy & Terms of Service (Sec III.2):** Accessible legal pages with genuine content relevant to the app (no empty placeholders) located in footer links. Mandatory for evaluation compliance.

3. Official Cybersecurity & Security Modules (Points Breakdown)

To reach the required 14 points (or earn bonus points), the following security-focused modules from Chapter IV should be targeted and owned by you:

Module Name	Category & Type	Points	Key Requirements & Security Implementation
-------------	-----------------	--------	--

WAF/ModSecurity + HashiCorp Vault	Cybersecurity (IV.5) <i>Major</i>	2 Pts	• Configure strict ModSecurity WAF rules (OWASP Core Rule Set) on Nginx proxy. • Deploy HashiCorp Vault for encrypted secrets management (API keys, DB credentials, JWT secrets).
Two-Factor Authentication (2FA)	User Mgmt (IV.3) <i>Minor</i>	1 Pt	• Implement TOTP 2FA (Google Authenticator, Authy). • QR code generation during setup + emergency backup recovery codes.
OAuth 2.0 Integration	User Mgmt (IV.3) <i>Minor</i>	1 Pt	• Remote auth via Google, GitHub, or 42 Intra. • Secure token exchange, PKCE state validation, profile mapping.
Advanced Permissions System (RBAC)	User Mgmt (IV.3) <i>Major</i>	2 Pts	• Fine-grained Role-Based Access Control (Admin, Recruiter, Candidate, Moderator). • Middleware-enforced permissions on all CRUD endpoints.
GDPR Compliance Features	Data & Analytics (IV.8) <i>Minor</i>	1 Pt	• Export user data in JSON/CSV format. • Self-service account deletion with confirmation & email notifications.
Secured Infrastructure (ELK/Grafana)	DevOps (IV.7) <i>Major/Minor</i>	1-2 Pts	• Enforce authenticated and TLS-encrypted access to Kibana and Grafana dashboards. • Centralized security audit logging.

4. Additional Proactive Hardening (Pro-Level Implementation)

Beyond mandatory rules, implementing these best practices will elevate your project to enterprise-grade security:

- **HTTP Security Headers:** Configure Nginx to return Strict-Transport-Security (HSTS), Content-Security-Policy (CSP), X-Frame-Options: DENY, X-Content-Type-Options: nosniff, and Referrer-Policy: strict-origin-when-cross-origin.
- **Session & JWT Security:** Store access tokens in HttpOnly, SameSite=Strict, Secure cookies to eliminate XSS token theft. Implement short lifespan (15m) + refresh token rotation.
- **Rate Limiting & Anti-Brute-Force:** Protect authentication endpoints (/auth/login, /auth/register, password reset) with Nginx limit_req or FastAPI slowapi rate limiting (e.g., max 5 attempts/min).
- **CORS & Origin Hardening:** Strictly restrict Cross-Origin Resource Sharing (CORS) to the exact frontend domain. Never use Access-Control-Allow-Origin: * with credentials.
- **Container Security & Least Privilege:** Use multi-stage Dockerfiles. Run containers under a non-root dedicated user (e.g., USER node or USER appuser). Scan images with trivy.
- **Automated Dependency & Static Analysis:** Add pip-audit, npm audit, and semgrep static analysis to GitHub Actions CI workflow to block vulnerable dependencies.

5. Skills Curriculum — What You Need to Learn

To fulfill this role with complete technical mastery and excel during peer evaluation defense, focus your study on these 6 core domains:

Domain	Key Concepts to Master	Practical Tools & Frameworks
Web Vulnerabilities	OWASP Top 10, SQL Injection, Stored/Reflected XSS, CSRF attacks, Broken Access Control, IDOR.	OWASP ZAP, Burp Suite Community, Semgrep, DMPurify.

Auth & Cryptography	Symmetric vs Asymmetric encryption, Password Hashing (Argon2, bcrypt), JWT structure, OAuth 2.0 flows, TOTP (RFC 6238).	PyJWT / Jose, Passlib, PyOTP, qrcode, Auth0 / Supabase Auth.
Network & WAF	TLS/SSL handshake, HTTPS certificates, Nginx reverse proxy, ModSecurity rules engine, OWASP Core Rule Set.	Nginx, ModSecurity v3, OWASP CRS, mkcert, certbot.
Secret Management	Key-value secret engines, Vault authentication, dynamic secrets, token leases, secret injection in runtime.	HashiCorp Vault CLI / API, HVAC (Python library).
Container Security	Docker security context, non-root users, Linux capabilities, image layer caching risks, Docker socket security.	Docker compose, Trivy vulnerability scanner, Hadolint.
Data Privacy & Compliance	GDPR principles (Right to be Forgotten, Data Portability, Consent), PII data encryption at rest.	SQLAlchemy encryption extensions, automated data export scripts.

6. Step-by-Step Security Action Plan

Execute your work across 4 structured phases to ensure complete coverage and seamless peer evaluation readiness:

Phase 1: Foundation & Mandatory Hardening (Week 1)

- 1. Set up SSL certificates and configure Nginx reverse proxy with HTTPS enforcement.
- 2. Audit `.env` & `env.example`` to ensure zero secrets in Git.
- 3. Verify backend password hashing (Argon2 / bcrypt) and double-sided validation.
- 4. Ensure Privacy Policy and Terms of Service pages exist with actual content.

Phase 2: Core Cybersecurity Modules (Weeks 2-3)

- 1. Deploy ModSecurity WAF container with OWASP Core Rule Set on Nginx.
- 2. Deploy HashiCorp Vault, configure secret engine, and migrate DB credentials & API keys.
- 3. Implement TOTP 2FA endpoints & UI modal (QR code scanning + verification).
- 4. Implement Role-Based Access Control (RBAC) middleware for route authorization.

Phase 3: Defensive Polish & Compliance (Week 4)

- 1. Configure security headers (HSTS, CSP, X-Frame-Options) and rate limiters.
- 2. Implement GDPR data export & account self-deletion features.
- 3. Harden Dockerfiles (non-root users, minimal images).
- 4. Run vulnerability scans with Trivy and fix all CRITICAL/HIGH findings.

Phase 4: Audit & Peer Defense Preparation (Pre-Evaluation)

- 1. Conduct full security audit & attempt penetration testing on login/signup/APIs.
- 2. Document WAF rules, Vault architecture, and security design in README.md.
- 3. Prepare quick live code walkthrough for peer evaluators (be ready to explain password hashing, WAF blocking, and Vault secret fetching).